

Chapter 5. Stateful stream processing

This chapter covers

- Processing multiple events from a stream by using state
- The most popular stream processing frameworks
- Using Apache Samza for detecting abandoned shopping carts
- Deploying a Samza job on Apache Hadoop YARN

In [chapter 3](#), we introduced the idea of processing continuous event streams and implemented a simple application that processed individual shopping events from the Nile website. The app we wrote did a few neat things: it read individual events from Kafka, filtered out bad input events, enriched the event with location information, and finally wrote the newly filtered and enriched event back out to Kafka.

[Chapter 3](#)'s app was relatively simple because it operated on only a single event at a time: it read each individual event off a Kafka topic, and then decided whether it would either filter the event (discard it), or enrich the event and write that enriched event back to a new Kafka topic. In the terminology introduced in [chapter 3](#), our app was performing *single-event processing*, whereby one input event generates zero or more output events, in contrast to what we call *multiple-event processing*, whereby one or more input events generates zero or more output events.

This chapter is all about multiple-event processing—or as we will start calling it (for reasons I'll explain soon)—*stateful stream processing*. In this chapter, we will write an application that generates outgoing events based on multiple incoming events. Continuing our employment at fictitious online retailer Nile, this time we will be striving to improve the online shopping experience, by detecting whenever a Nile shopper has abandoned their shopping cart.

As we hinted in [chapter 3](#), processing multiple events at a time is more complex than processing single events. We need to maintain some form of state to keep track of shopper behavior across multiple events; this brings with it attendant challenges, such as distributing the processing and the state safely over multiple servers. To meet these challenges, we will introduce *stream processing frameworks* at a high level.

We will implement our abandoned shopping cart detector in Java by using the Apache Samza stream processing framework. Samza is not the most featureful or well-known stream processing framework; the API it provides is relatively basic, slightly reminiscent of Hadoop's original Map-Reduce API in Java. But Samza's simplicity is also a strength: it will make

it easier for us to see and understand the essential stateful nature of this stream processing job. Writing this job with Samza should give you the confidence to try out the newer and “buzzier” frameworks like Spark Streaming and Flink.

But first we will introduce our new business challenge and outline its stream processing requirements.

5.1. Detecting abandoned shopping carts

Remember: we currently work for a sells-everything e-commerce website, called Nile. The management team at Nile wants the company to become much more dynamic and responsive by reacting to customer behavior in a timely and effective fashion. This will be a great opportunity for us to implement *stateful stream processing*.

5.1.1. What management wants

As part of their goal of creating a more dynamic and responsive business, the management team at Nile has identified a key opportunity around identifying and reacting to shopper-abandoned shopping carts. A shopping cart is defined as *abandoned* when a shopper adds products to their shopping cart but doesn’t continue through to checkout.

For online retailers like Nile, it is worthwhile contacting shoppers who have abandoned their carts and asking if they would like to complete their orders. Timing is everything here: it’s important to identify and react to abandoned shopping carts quickly, but not so quickly that a shopper feels pestered or rushed; UK handbag company Radley released a study showing that 30 minutes after abandonment is the optimal time to get in touch.^[1]

¹

Details of this study are available at <http://d34w0339mx0ifp.cloudfront.net/global/images/uploads/2013/11/Radley-Client-Story.pdf>.

Typically, an online retailer responds to an abandoned shopping cart by emailing the shopper, or by showing the shopper retargeting ads on other websites, but we don’t need to worry about the exact response mechanism. The important thing is to define a new event, *Shopper abandons cart*, and generate one of these new events whenever we detect an abandoned cart. Nile’s data engineers can then read these new events from the relevant stream and decide how to handle them.

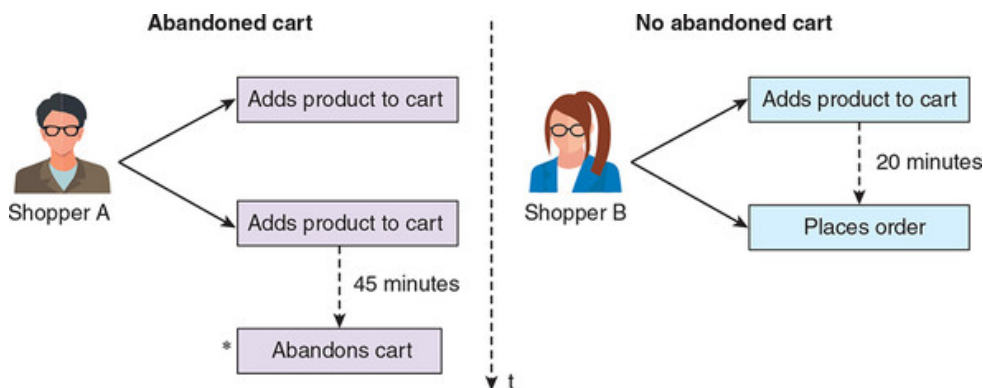
5.1.2. Defining our algorithm

How do we detect an abandoned shopping cart? For the purposes of this chapter, let's use a simple algorithm:

- Our shopper adds a product to the cart.
- Derive a *Shopper abandons cart* event if 30 minutes pass without one of the following occurring:
 - Our shopper adding any further products to cart
 - Our shopper placing an order
- If our shopper adds a further product to the cart during the 30 minutes, restart the timer.
- If our shopper places an order during the 30 minutes, clear the timer.

[Figure 5.1](#) presents two examples of applying this algorithm. This algorithm is not a particularly sophisticated one, but it should help Nile get started tackling its abandoned shopping carts problem, and it can always be refined further later.

Figure 5.1. On the left side, Shopper A has added two products to the shopping cart, and then 45 minutes have passed without any further activity, so we can derive a *Shopper abandons cart* event. On the right side, Shopper B has added a product to the shopping cart and checked out within 20 minutes, not abandoning the cart.



5.1.3. Introducing our derived events stream

When an abandoned cart is detected, a new *Shopper abandons cart* event is generated, per the preceding algorithm. But where should we write this new event to? We have two options:

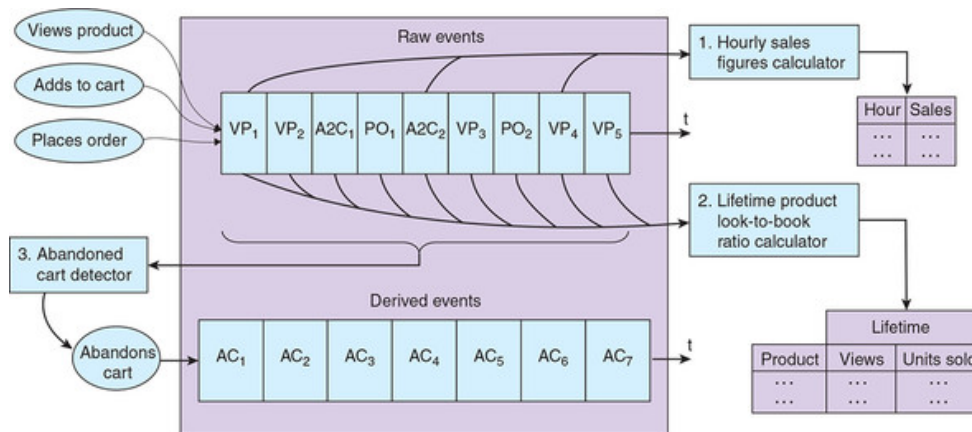
- We write it back to the `raw-events-ch05` topic, colocating it with the original shopper events that were fed into the algorithm.
- We write it to a new Kafka topic, called `derived-events-ch05`.

Both approaches have their merits. Writing the new event back to `raw-events-ch05` is simple to reason about: all events can now be found in a single stream, regardless of the process that generated them. On the other hand, writing to a separate stream, `derived-events-ch05`, makes it clear that these new events are *second-order events*, derived from the

original events. A data engineer who cares only about this event (for example, to send abandoned shopping cart emails) can read this new stream and ignore the original stream.

For this chapter, we will opt for the second approach and write our *Shopper abandons cart* events to a new Kafka topic, called `derived-events-ch05`. [Figure 5.2](#) shows our new stream processing job, reading events from the `raw-events-ch05` topic, and writing new events to a `derived-events-ch05` topic.

Figure 5.2. Our abandoned shopping cart detector consumes events from the `raw-events-ch05` topic in Kafka and generates new *Shopper abandons cart* events to write back to a new Kafka topic, called `derived-events-ch05`.



There is a lot to take in here: we have defined an algorithm to meet Nile's goals around abandoned shopping carts and introduced a second Kafka topic to receive the new events. Before we go any further, we need to model the various event types that we will be dealing with here.

5.2. Modeling our new events

Remember back in [chapter 2](#), when we introduced our first e-commerce-related event for Nile, *Shopper views product*? Drawing on our standard definition of an event as *subject-verb-object*, we can identify three further discrete events required for tracking abandoned carts:

- ***Shopper adds item to cart***— The shopper adds one of those products to the shopping basket. A product is added to the basket with a quantity of one or more attached.
- ***Shopper places order***— The shopper checks out, paying for the items in the shopping basket.
- ***Shopper abandons cart***— The derived event itself, representing the act of cart abandonment by the shopper.

Before we jump into writing our stream processing application, let's first *model* these three new event types. Although it's tempting to skip this step

and dive into the coding, having clear definitions of the events that we will be working with will save us a lot of time and should prevent us from getting stuck down any coding cul de sacs.

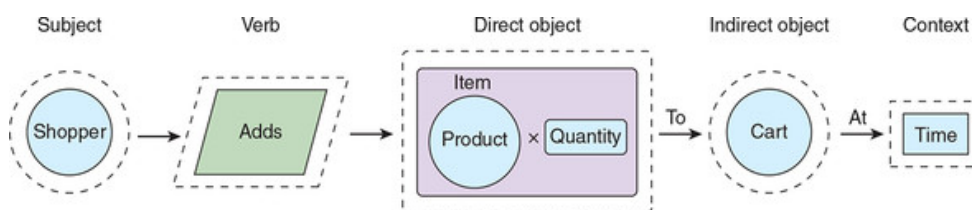
5.2.1. Shopper adds item to cart

The *Shopper adds item to cart* event involves a Nile shopper adding a product to their shopping cart (also known as a *shopping basket*), specifying a quantity of that product as they do this. Let's break out the various components here:

- **Subject:** Shopper
- **Verb:** Adds
- **Direct object:** Item (consisting of product and quantity)
- **Indirect object:** Cart
- **Context:** Timestamp of this event

[Figure 5.3](#) illustrates these components.

Figure 5.3. Our shopper (again, subject) adds (verb) an item, consisting of a product and its quantity (direct object) to the shopping cart (indirect, aka prepositional, object) at a given time (context).



5.2.2. Shopper places order

This event sounds simple, and it is:

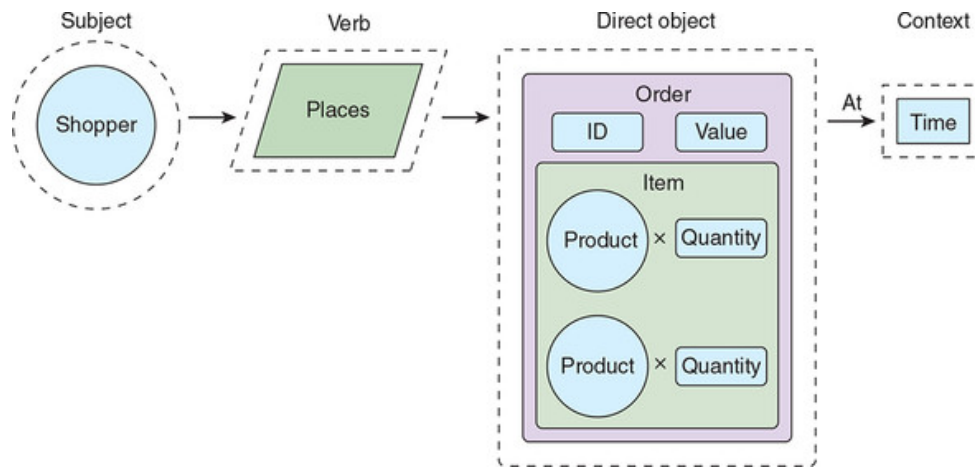
- **Subject:** Shopper
- **Verb:** Places
- **Direct object:** Order
- **Context:** Timestamp of this event

The slight complexity is in modeling the order. This is a complicated entity: it needs to contain an order ID and a total order value, plus a list of items that were purchased in the order; each item should be a product and the quantity of that product ordered.

Putting it all together, we get the event drawn in [figure 5.4](#).

Figure 5.4. Our shopper (always the subject) places (verb) an order (direct object) at a given time (context). The order contains ID and

value attributes, plus an array of order items, each consisting of a product and quantity of that product.



5.2.3. Shopper abandons cart

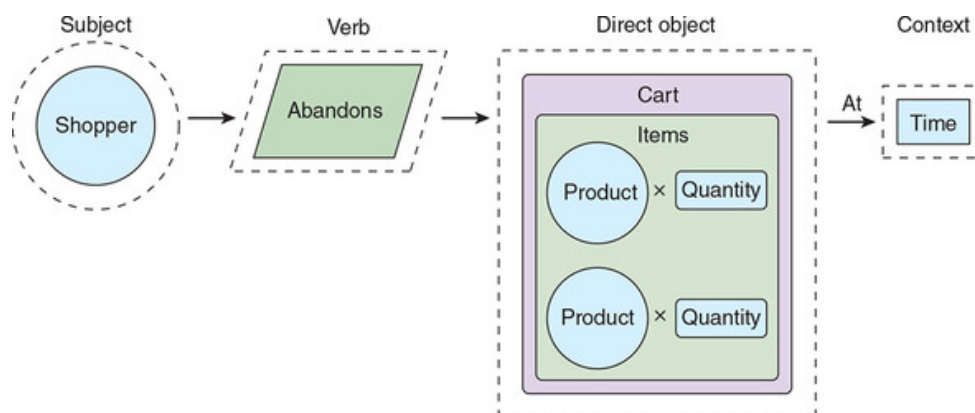
Now we come to our derived event. By *derived*, we mean an event that we are generating ourselves in our stream processing application, as opposed to an incoming raw event that we are simply consuming.

This event looks like this:

- **Subject:** Shopper
- **Verb:** Abandons
- **Direct object:** Cart (consisting of multiple items, each of a product and quantity)
- **Context:** Timestamp of this event

Our fourth and final event for this chapter is illustrated in [figure 5.5](#).

Figure 5.5. Our shopper (subject) abandons (verb) their shopping cart (direct object) at a given time (context). The shopping cart contains an array of order items, each consisting of a product and quantity of that product.



This completes the set of Nile e-commerce events required for our abandoned shopping cart detector. In the next section, we will look at our final building block: stateful stream processing.

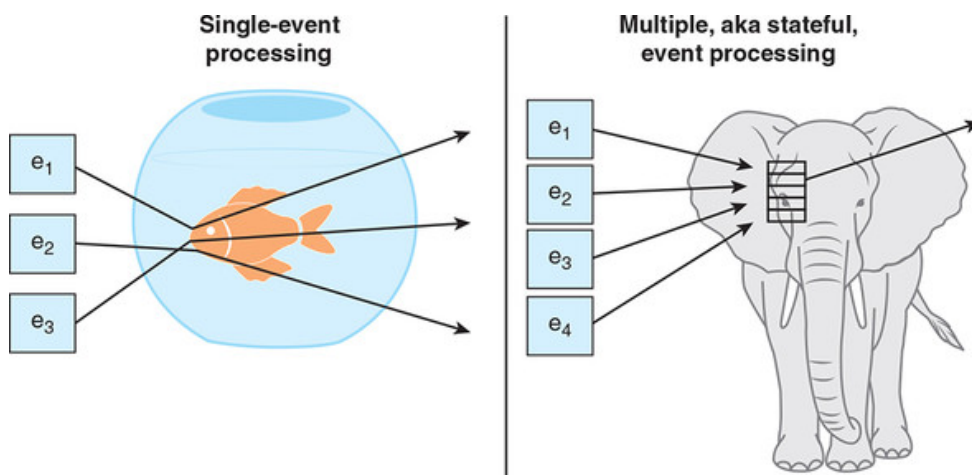
5.3. Stateful stream processing

To detect abandoned shopping carts for Nile, we have to do some *stream processing* on the incoming events. Clearly, it's a little more complex than [chapter 3](#), where we were able to work on only a single event at a time: our algorithm expects us to understand the flow of multiple events in a sequence over time. The key building block for processing multiple events like this is state.

5.3.1. Introducing state management

If single-event processing is a little like being a goldfish, multiple-event processing is like being an elephant. The goldfish stream processor can forget each event as soon as it has processed it, whereas the elephant stream processor requires a memory of prior events in order to generate its output. We can call this memory of prior events *state* and say that multiple-event processing is therefore *stateful*. [Figure 5.6](#) illustrates this slightly tortuous metaphor.

Figure 5.6. Our goldfish-like single-event processor can process each event and then immediately forget about it. By contrast, our elephant-like stateful event processor must remember multiple events to perform aggregations, sorting, pattern detection, and suchlike.

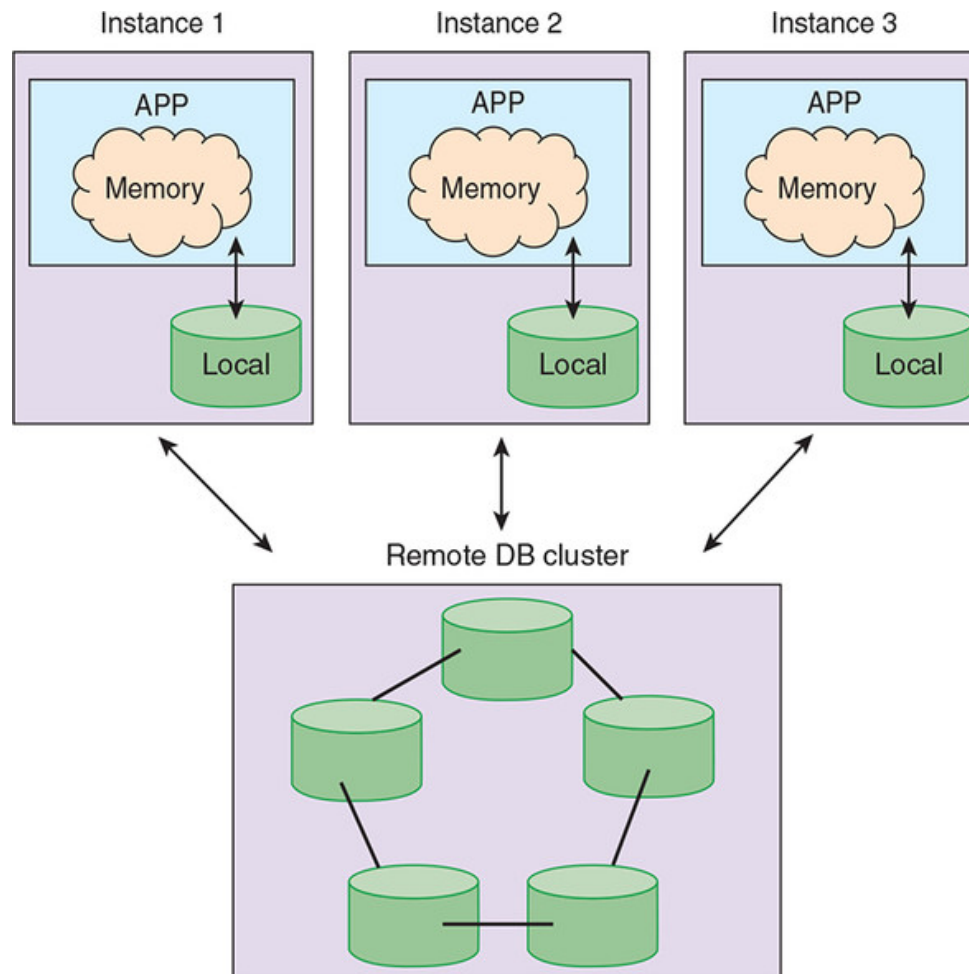


What kind of technology should we be using to implement this state in our stream processing application? There are lots of options, but they largely boil down to three:

- **In-process memory**— A mutable variable or variables available inside the stream processing application's own code
- **Local data store**— Some kind of simple data store (for example, LevelDB, RocksDB, or SQLite) that is local to the server or container running this instance of the stream processing application
- **Remote data store**— Some kind of database server or cluster (for example, Cassandra, DynamoDB, or Apache HBase) hosted remotely

[Figure 5.7](#) depicts these three options. Regardless of the specific approach, the important thing to understand is that processing multiple events needs state.

Figure 5.7. A distributed stream-processing application (here with three instances) can keep state in in-process memory, in an instance-local data store, or in a remote data store—or even in a combination of all three.



5.3.2. Stream windowing

We know that we need some form of state to process multiple events, but what exactly is going to be kept in that state? When we think of data processing, we are almost always thinking of processing a *bounded set* of data—a collection of data points that are of a somehow finite size. After all, even an elephant has a limit on how much they can remember. Here are some examples of data queries and transformations from a bounded set, with the bounded set in *italics*:

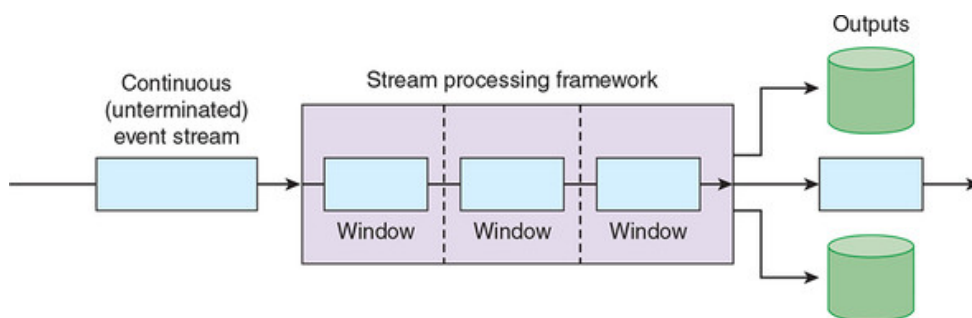
- Who is the fastest marathon runner *since records began*?
- How many T-shirts did we give away *in January 2018*?
- What are our *year-to-date* revenues?

Unfortunately for our purposes, a continuous event stream is *unterminated*, meaning that it has no end (and possibly no beginning either), so the first challenge in multiple-event processing is to come up with a sensible

way of bounding a continuous event stream. We typically solve this problem by applying a *processing window* to our event stream.

At its simplest, processing on a continuous event stream can be put into discrete windows by using a timer, or heartbeat, function that runs at a regular interval. This timer function can contain custom code to apply whatever window or windows makes sense for the use case. [Figure 5.8](#) illustrates this idea of slicing an unending event stream into specific windows for further processing.

Figure 5.8. A stream processing framework typically applies windowing to a continuous (unterminated) event stream to process it and generate meaningful outputs.



In the case of our abandoned shopping cart, the processing window is well-defined: we are looking for shopping carts that have been abandoned by their owner for at least 30 minutes. We don't want to leave an abandoned shopping cart much longer than the required 30 minutes, so a timer checking each cart every 30 seconds would be ideal.

If stream windowing doesn't make a lot of sense yet, don't worry; we will be exploring it in more detail when we start building our abandoned cart detector. The key takeaway for now is that stream windowing is an important building block for stateful stream processing.

5.3.3. Stream processing frameworks and their capabilities

All this talk of processing multiple events, maintaining state, and creating stream windows probably sounds like a lot of work! To solve these and other problems in a repeatable, reliable way, a handful of *stream processing frameworks* have emerged. These frameworks exist to abstract away the mechanics of running stateful stream processing at scale, leaving developers to focus on the actual business logic that their jobs require.

Stream processing frameworks exhibit some or all of the following capabilities:

- **State management**— State is a key building block for processing multiple events at a time. Stream processing frameworks give you the option of storing state in some or all of the following: in-memory, in a lo-

cal filesystem, or in a dedicated key-value store such as RocksDB or Redis.

- **Stream windowing**— As described previously, a stream processing framework provides one or more ways of expressing a *bounded window* for the event processing. This is typically time-based, although sometimes it can be based on a count of events instead.
- **Delivery guarantees**— All stream processing frameworks pessimistically track their progress, to ensure that every event is processed *at least once*. Some of these systems go further, adding in transactional guarantees to ensure that each event is processed only *exactly once*.
- **Task distribution**— A high-volume event stream consists of multiple Kafka topics or Amazon Kinesis streams, and requires multiple instances of the job, or tasks, to process it. Most stream processing frameworks are designed to be run on a sophisticated third-party scheduler such as Apache Mesos or Apache Hadoop YARN; some stream processing frameworks are also *embeddable*, meaning that they can be added as a library to a regular application (requiring bespoke scheduling).
- **Fault tolerance**— Failures happen regularly in the kind of large-scale distributed systems required to process high-volume event streams, and most frameworks have built-in mechanisms to automatically recover from these failures. Fault tolerance typically involves a distributed backup of either the incoming events or the generated state.

Let’s see how some of the most popular stream processing frameworks relate to these capabilities.

5.3.4. Stream processing frameworks

A variety of stream processing frameworks have emerged over the past few years, with most of the popular ones having been successfully incubated by the Apache Software Foundation. [Table 5.1](#) introduces five of the most widely used frameworks and lays out their design relative to the capabilities introduced in the preceding section.

Table 5.1. A nonexhaustive list of stream processing frameworks

Capability	Storm	Samza	Spark Streaming	Kafka Streams	Flink
State management	In-memory, Redis	In-memory, RocksDB	In-memory, filesystem	In-memory, RocksDB	In-memory, filesystem, RocksDB

Capability	Storm	Samza	Spark Streaming	Kafka Streams	Flink
Stream windowing	Time-, count-based	Time-based	Time-based (microbatch)	Time-based	Time-, count-based
Delivery guarantees	At least once, exactly once (Trident)	At least once	At least once, exactly once	At least once, exactly once	Exactly once
Task distribution	YARN or Mesos	YARN or embeddable	YARN or Mesos	Embeddable	YARN or Mesos
Fault tolerance	Record acks	Local, distributed snapshots	Checkpoints	Local, distributed snapshots	Distributed snapshots

Let's go through each of these stream processing frameworks briefly.

Apache Storm

Apache Storm was the first of the “new wave” of stream processing frameworks. Storm was written by Nathan Marz at BackType; Twitter acquired BackType and open sourced Storm in 2011. As a pioneering piece of technology, Storm did things slightly differently than its successors:

- Fault tolerance is achieved by record acknowledgments with upstream record backups, rather than state snapshots or checkpoints.
- Rather than using a third-party scheduler, Storm originally created its own (Nimbus)—although Storm supports both YARN and Mesos now.
- Storm introduced a separate library, Storm Trident, to support exactly-once processing.

Storm was popularized by the book *Big Data* by Nathan Marz and James Warren (Manning) and continues to be widely used; Twitter's successor system, Heron, and Apache Flink (covered later in this section) both offer Storm-compatible APIs to ease adoption.

Apache Samza

Apache Samza is a stateful stream-processing framework from the team at LinkedIn who also originated Apache Kafka. As such, Samza has tight integration with Apache Kafka, using Kafka for the backing up of the state

held in RocksDB databases. Samza has a relatively low-level API, letting you interact directly with Samza's stream windowing and state management features. Samza promotes building any required complex *stream-processing topology* out of many simpler Samza jobs, reading from and writing back to separate Kafka topics.

Samza jobs were originally designed to be run from the YARN application scheduler; more recently, Samza can also be embedded as a library inside a regular application, letting you choose your own scheduler (or no scheduler). Samza has also outgrown its original inter-dependence with Kafka, now also supporting Amazon Kinesis.

Samza continues to evolve, is a great teaching tool for stream processing, and is used by major companies. But Kafka Streams (covered later in this section) has perhaps stolen some of Samza's thunder.

Apache Spark Streaming

Spark Streaming is an extension of the Apache Spark project for working with continuous event streams. Technically speaking, Spark Streaming is a *microbatch* processing framework, not a *stream* processing framework: Spark Streaming slices the incoming event stream into microbatches, which it then feeds to the standard Spark engine for processing.

Spark Streaming's microbatching gives us exactly-once processing "for free" and lets us reuse our existing Spark experience and code, should we have it. But it comes at a cost: the microbatching increases latency and reduces our flexibility around stream windowing and scaling.

Spark Streaming is not tied to any one stream technology: almost any incoming stream technology can be sliced and fed into Spark in microbatches; out-of-the-box Spark Streaming supports Flume, Kafka, Amazon Kinesis, files, and sockets, and you can write your own *custom receiver* if you like.

For readers interested in digging further into Spark Streaming, *Spark in Action* by Petar Zečević and Marko Bonaći, and *Streaming Data* by Andrew G. Psaltis, both published by Manning, are great resources.

Apache Kafka Streams

Kafka Streams is a stream processing library for Apache Kafka, conceived by the LinkedIn Kafka team after they decamped LinkedIn for Kafka startup Confluent. Kafka Streams has shared conceptual DNA with Samza, but is on a somewhat different trajectory:

- Kafka Streams was designed from the start as a library to be embedded in your own application code; it doesn't support any existing scheduler such as YARN or Mesos.
- Kafka Streams is closely tied to Kafka; Kafka Streams uses Kafka for all aspects of its state management, fault tolerance, and delivery guarantees; you cannot use Kafka Streams with any other stream technology.
- Although Kafka Streams does expose a low-level API, it also has a higher-level query API that will be more familiar to Spark users.

Although a young project, Kafka Streams is quickly gaining serious developer mindshare as a first-class citizen within the Kafka ecosystem. Confluent is positioning Kafka Streams as a toolkit for asynchronous or event-driven microservices, as distinct from the more analytics or data science use cases that Spark is noted for.

Another great Manning book to dig into Kafka Streams is *Kafka Streams in Action* by William P. Bejeck Jr.

Apache Flink

Apache Flink is a relative newcomer but rapidly emerging as a credible challenger to Spark and Spark Streaming. Unlike Spark, which takes a batch model and handles streaming use cases via microbatching, Flink was built from the start as a streaming engine: it handles batch data by treating it as a bounded stream, one with a beginning and an end.

Like Spark, Flink is not tied to Kafka. Flink supports various other data sources and sinks, including Amazon Kinesis. Flink has sophisticated stream windowing capabilities, and supports exactly-once processing and a rich query API; Flink is also closely following the Apache Beam project, which aims to provide a standard, full-featured API for interacting with stream processing frameworks.

This concludes our introduction to five major stream-processing frameworks, but how do we choose between these for our work at Nile?

5.3.5. Choosing a stream processing framework for Nile

As mentioned at the beginning of this chapter, we are going to use Apache Samza to build our abandoned cart detector. Samza is a full-blown stream processing framework, but unlike the other tools we've introduced, Samza makes no effort to abstract or otherwise hide away the stateful nature of the event processing involved: with Samza, you will be working directly with a simple key-value data store to keep track of events that you have already processed.

What is a *key-value store*? It's a super-simple database in which you store a single *value* (an array of bytes) against an individual unique *key* (also represented by an array of bytes); the key and the value can be any value that you like. A key-value store is a crude tool, but an effective one. It will give us the state we need to track shopper behavior across multiple events. Samza uses RocksDB, an embeddable persistent (versus in-memory only) key-value store, created by Facebook.

Samza has other tricks up its sleeves. The core of a Samza job consists of just two functions:

- `process()`—Called for each incoming event
- `window()`—Called on a regular, configurable interval

Both functions have access to the key-value store, plus any mutable state defined in the job; this lets them effectively communicate with each other.

If other stream processing frameworks are fishing sticks, Samza is a fishing rod and tackle: the way that Apache Samza exposes the underlying key-value store as a first-class entity for direct manipulation makes Samza an excellent teaching tool for stateful stream processing. Similarly, the no-nonsense `process()` and `window()` functions make it easy to visualize exactly what is going on in our stream processing job at any moment.

5.4. Detecting abandoned carts

A recap: we are writing a stateful stream-processing job that will search the incoming stream, looking for a specific pattern of events, and when that pattern is found, the job will emit a new event, *Shopper abandons cart*. We are going to implement this job in Apache Samza. Let's get started.

5.4.1. Designing our Samza job

To detect abandoned carts, we need a way of encapsulating shopper behavior in Samza's key-value store. We must represent each observed shopper's current *state* in the key-value store and keep this state up-to-date so that we can generate a *Shopper abandons cart* event as soon as that behavior is detected.

With a key-value store, it is completely up to us how we design our *key-space*—the meaning and layout of keys (and thus their values) in the database. In this case, we can use a pair of namespaced keys to maintain our shopper's current state in Samza, like so:

- `<shopper>-ts` should be kept up-to-date with the timestamp at which our job saw the most recent *Shopper adds item to cart* event for the given shopper.
- `<shopper>-cart` should be kept up-to-date with the current contents of the shopper's cart, based on aggregating all *Shopper adds item to basket* events.

It's important that we don't pester Nile's customers about abandoned carts. So, we should delete both keys from Samza (effectively "resetting" tracking for this user) as soon as we see a *Shopper places order* event; this should be done regardless of which products were purchased in that order. We should also delete these keys immediately after sending a new *Shopper abandons cart* event, so we don't send the event twice.

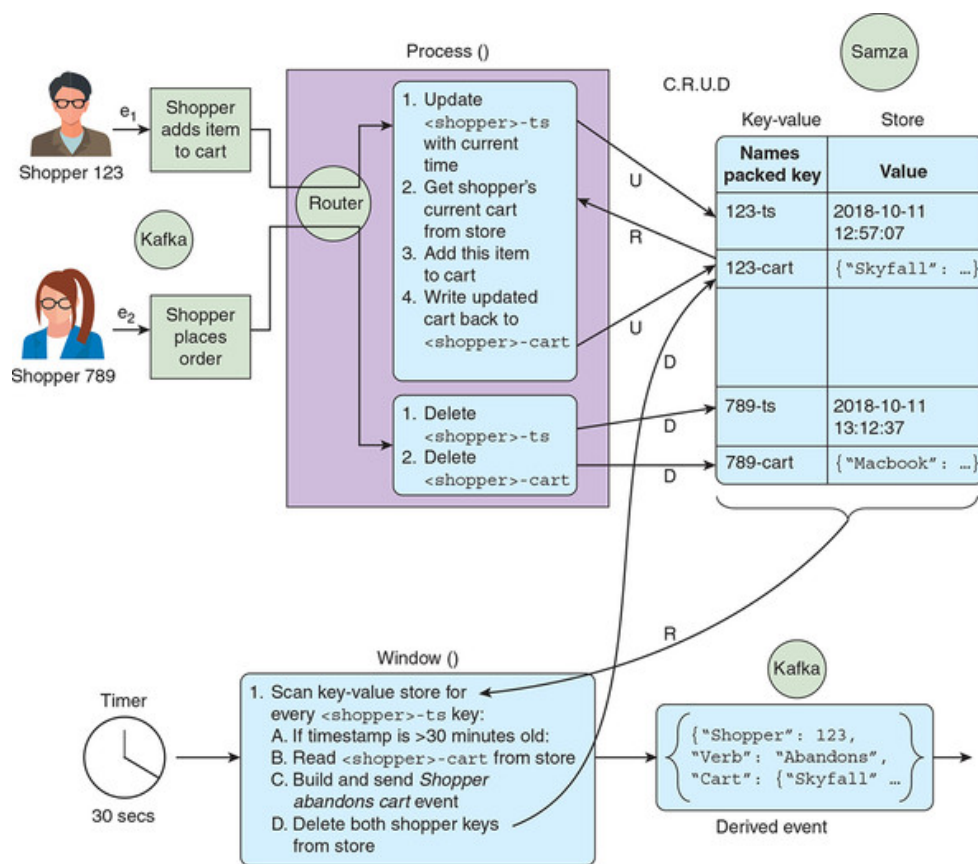
Putting this together, we have two events that we are interested in for our `process()` function:

- ***Shopper adds item to cart***— For tracking the shopper's cart state and the time when they were last active with their cart
- ***Shopper places order***— For telling us to "reset" tracking for this user

For our `window()` function, we will scan through the whole key-value store, looking for shoppers who were last active more than 30 minutes ago, based on the `<shopper>-ts` values. Whenever we find one, we will generate a new *Shopper abandons cart* event, containing the cart contents for this shopper as fetched from `<shopper>-cart`.

There's a lot to take in here. Don't worry—it should become clearer after you see the Java task code. In the meantime, [figure 5.9](#) sets out the overall design of this job.

Figure 5.9. Our `process()` function parses incoming events from Nile's shoppers and updates the Samza key-value store to track the shoppers' behavior. The `window()` function then runs regularly to scan the key-value store and identify shoppers who have abandoned their carts. A *Shopper abandons cart* event is then emitted for those shoppers.



5.4.2. Preparing our project

Unfortunately, a lot of ceremony exists around setting up a new Samza project, which is a distraction from the actual stream processing we want to implement. Happily, we can skip most of this ceremony by starting from the Samza's team's own Hello World project. This will serve as our cuckoo's nest. Let's check out the project like so:

```
$ git clone https://git.apache.org/samza-hello-samza.git nile-carts
$ cd nile-carts
$ git checkout f488927
Note: checking out 'f488927'.
...
```

Checking out the specific commit ensures that we are using the release of Hello World that works with Samza version 0.14.0.

Let's clean up the folder structure a little:

```
$ rm wikipedia-raw.json
$ rm src/main/config/*.properties
$ rm -rf src/main/java/samza
```

Apache projects use a Maven plugin called Rat. Using this with our project will cause problems, so let's delete this from the Maven pom.xml file:

```
$ sed -i '' '257,269d' pom.xml
```

Finally, let's rename the artifact to prevent confusion later. Edit the pom.xml file in the root and change the artifactId element on line 29 from hello-samza to the following value:

```
<artifactId>nile-carts</artifactId>
```

Similarly, edit the file src/main/assembly/src.xml and replace the include element on line 69 from hello-samza to the following value:

```
<include>org.apache.samza:nile-carts</include>
```

That's enough feathering of the nest. Let's move on to configuring our job.

5.4.3. Configuring our job

Although Samza jobs are written in Java, they are typically assembled with a Java properties file that tells Samza exactly how to run the file. Let's create this configuration file now. Open your editor and create a file at this path:

```
src/main/config/nile-carts.properties
```

Populate your new properties file with the following configuration.

Listing 5.1. nile-carts.properties

```
# Job
job.factory.class=org.apache.samza.job.yarn.YarnJobFactory
job.name=nile-carts
job.coordinator.system=kafka
job.coordinator.replication.factor=1

# YARN
yarn.package.path=file://${basedir}/target/${project.artifactId}-
➡ ${pom.version}-dist.tar.gz

# Task
task.class=nile.tasks.AbandonedCartStreamTask
task.inputs=kafka.raw-events-ch05
task.window.ms=30000

# Serializers
serializers.registry.json.class=org.apache.samza.serializers.
JsonSerdeFactory
serializers.registry.string.class=org.apache.samza.serializers.
```

➡ StringSerdeFactory

```
# Systems
systems.kafka.samza.factory=org.apache.samza.system.kafka.KafkaSystemFact
systems.kafka.samza.msg.serde=json
systems.kafka.consumer.zookeeper.connect=localhost:2181/
systems.kafka.producer.bootstrap.servers=localhost:9092
systems.kafka.consumer.auto.offset.reset=largest
systems.kafka.producer.metadata.broker.list=localhost:9092
systems.kafka.producer.producer.type=sync
systems.kafka.producer.batch.num.messages=1

# Key-value storage
stores.nile-carts.factory= org.apache.samza.storage.kv
    .RocksDbKeyValueStorageEngineFactory
stores.nile-carts.changelog=kafka.nile-carts-changelog
stores.nile-carts.changelog.replication.factor=1
stores.nile-carts.key.serde=string
stores.nile-carts.msg.serde=string
stores.nile-carts.write.batch.size=0
stores.nile-carts.object.cache.size=0
```

To go through each of the configuration blocks in turn:

- The `Job` section tells us that this is the `nile-carts` job, and will be run on YARN.
- The `YARN` section defines the location of our Samza job package for YARN.
- The `Task` section specifies the Java class that will contain the job's stream processing logic, which we'll write in the next section. The Java class will process events coming from the Kafka topic `raw-events-ch05`, and it will have a processing window of 30 seconds (30,000 milliseconds).
- The `Serializers` section declares two *serdes* (serializers-deserializers) that will let us read and write strings and JSON in our job.
- The `Systems` section configures Kafka for our job. We specify that events we consume and produce from Samza will be in JSON format.
- The `Key-value storage` section configures our local state. Our keys and values in the key-value store will all be strings.

And that's it for the configuration—now on to the code.

5.4.4. Writing our job's Java task

Remember that this Samza job needs to generate a stream of well-structured *Shopper abandons cart* events. Rather than jumping straight into our task code, let's first create a base `Event` class, and then extend it for our `AbandonedCartEvent`.

In your editor, add the code in the following listing into a file at this path:

```
src/main/java/nile/events/Event.java
```

Listing 5.2. Event.java

```
package nile.events;

import org.joda.time.DateTime;           1
import org.joda.time.DateTimeZone;
import org.joda.time.format.DateTimeFormat;
import org.joda.time.format.DateTimeFormatter;
import org.codehaus.jackson.map.ObjectMapper;

public abstract class Event {           2

    public Subject subject;
    public String verb;
    public Context context;

    protected static final ObjectMapper MAPPER = new ObjectMapper();
    protected static final DateTimeFormatter EVENT_DTF = DateTimeFormat
        .forPattern("yyyy-MM-dd'T'HH:mm:ss").withZone(DateTimeZone.UTC);

    public Event(String shopper, String verb) {
        this.subject = new Subject(shopper);
        this.verb = verb;
        this.context = new Context();
    }

    public static class Subject {
        public final String shopper;           3

        public Subject() {
            this.shopper = null;
        }

        public Subject(String shopper) {
            this.shopper = shopper;
        }
    }

    public static class Context {
        public final String timestamp;

        public Context() {
            this.timestamp = EVENT_DTF.print(
                new DateTime(DateTimeZone.UTC)); 4
        }
    }
}
```

- 1 Using the JodaTime library
- 2 Our abstract Event class. Contains subject, verb, and context fields.
- 3 Our subject is a shopper, identified by a cookie value in a string.
- 4 Our context's timestamp will be set when the event is constructed.

Our abstract Event class is just a helper for modeling an event that conforms to the structure of *subject-verb-object*. We can extend it to create the AbandonedCartEvent class that our job will use to send well-formed *Shopper abandons cart* events. Stub a file at this path:

```
src/main/java/nile/events/AbandonedCartEvent.java
```

Add in the code in the following listing.

Listing 5.3. AbandonedCartEvent.java

```
package nile.events;

import java.io.IOException;
import java.util.List;
import java.util.ArrayList;
import java.util.Map;
import org.joda.time.DateTime;
import org.joda.time.DateTimeZone;
import org.codehaus.jackson.type.TypeReference;
import nile.events.Event;

public class AbandonedCartEvent extends Event {
    public final DirectObject directObject;

    public AbandonedCartEvent(String shopper, String cart) {
        super(shopper, "abandon");
        this.directObject = new DirectObject(cart);
    }

    public static final class DirectObject {
        public final Cart cart;

        public DirectObject(String cart) {
            this.cart = new Cart(cart);
        }
    }

    public static final class Cart {

        private static final int ABANDONED_AFTER_SECS = 1800;

        public List<Map<String, Object>> items =
            new ArrayList<Map<String, Object>>();
```



```

    public Cart(String json) {
        if (json != null) {
            try {
                this.items = MAPPER.readValue(json,
                    new TypeReference<List<Map<String, Object>>>() {});
            } catch (IOException ioe) {
                throw new RuntimeException("Problem parsing JSON cart", ioe);
            }
        }
    }

    public void addItem(Map<String, Object> item) {                2
        this.items.add(item);
    }

    public String asJson() {                                       3
        try {
            return MAPPER.writeValueAsString(this.items);
        } catch (IOException ioe) {
            throw new RuntimeException("Problem writing JSON cart", ioe);
        }
    }

    public static boolean isAbandoned(String timestamp) {         4
        DateTime ts = EVENT_DTF.parseDateTime(timestamp);
        DateTime cutoff = new DateTime(DateTimeZone.UTC)
            .minusSeconds(ABANDONED_AFTER_SECS);
        return ts.isBefore(cutoff);
    }
}

```

- **1 We will classify a cart as abandoned if no event activity occurs for 30 minutes.**
- **2 Adds an item to the cart. Note that we don't bother "de-duplicating" multiple additions of the same product.**
- **3 Converts the items in our cart to JSON, ready for writing to the key-value store**
- **4 Checks whether the supplied timestamp counts as abandoned (if more than 30 minutes old)**

Our `AbandonedCartEvent` gives us an easy way of generating a new *Shopper abandons cart* event, ready for our Samza job to emit whenever an abandoned cart is detected. The most interesting aspect is the `Cart` inner class, which contains a helper method used to add items to a cart whenever a *Shopper adds item to cart* event occurs. Meanwhile, the `isAbandoned()` helper function will tell us whether a given shopping cart has been abandoned.

With our *Shopper abandons cart* event defined, we are now ready to write the Java `StreamTask` that makes up the core of our Samza job. Back in your editor, create a file at this path:

```
src/main/java/nile/tasks/AbandonedCartStreamTask.java
```

And for the last time in this chapter, add in the code in the following listing.

Listing 5.4. AbandonedCartsStreamTask.java

```
package nile.tasks;

import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;
import java.util.Set;
import org.apache.samza.config.Config;
import org.apache.samza.storage.kv.KeyValueStore;
import org.apache.samza.storage.kv.KeyValueIterator;
import org.apache.samza.storage.kv.Entry;
import org.apache.samza.system.IncomingMessageEnvelope;
import org.apache.samza.system.OutgoingMessageEnvelope;
import org.apache.samza.system.SystemStream;
import org.apache.samza.task.InitableTask;
import org.apache.samza.task.MessageCollector;
import org.apache.samza.task.StreamTask;
import org.apache.samza.task.TaskContext;
import org.apache.samza.task.TaskCoordinator;
import org.apache.samza.task.WindowableTask;
import nile.events.AbandonedCartEvent;
import nile.events.AbandonedCartEvent.DirectObject.Cart;

public class AbandonedCartStreamTask
    implements StreamTask, InitableTask, WindowableTask {

    private KeyValueStore<String, String> store;

    public void init(Config config, TaskContext context) {
        this.store = (KeyValueStore<String, String>)
            context.getStore("nile-carts");
    }

    @SuppressWarnings("unchecked")
    @Override
    public void process(IncomingMessageEnvelope envelope,
        MessageCollector collector, TaskCoordinator coordinator) {

        Map<String, Object> event =
            (Map<String, Object>) envelope.getMessage();
        String verb = (String) event.get("verb");
```

```

String shopper = (String) ((Map<String, Object>)
    event.get("subject")).get("shopper");

if (verb.equals("add")) {
    String timestamp = (String) ((Map<String, Object>)
        event.get("context")).get("timestamp");

    Map<String, Object> item = (Map<String, Object>)
        ((Map<String, Object>) event.get("directObject")).get("item");
    Cart cart = new Cart(store.get(asCartKey(shopper)));
    cart.addItem(item);

    store.put(asTimestampKey(shopper), timestamp);
    store.put(asCartKey(shopper), cart.asJson());

} else if (verb.equals("place")) {
    resetShopper(shopper);
}

@Override
public void window(MessageCollector collector,
    TaskCoordinator coordinator) {

    KeyValueIterator<String, String> entries = store.all();
    while (entries.hasNext()) {
        Entry<String, String> entry = entries.next();
        String key = entry.getKey();
        String value = entry.getValue();
        if (isTimestampKey(key) && Cart.isAbandoned(value)) {
            String shopper = extractShopper(key);
            String cart = store.get(asCartKey(shopper));

            AbandonedCartEvent event =
                new AbandonedCartEvent(shopper, cart);
            collector.send(new OutgoingMessageEnvelope(
                new SystemStream("kafka", "derived-events-ch05"), event));

            resetShopper(shopper);
        }
    }
}

private static String asTimestampKey(String shopper) {
    return shopper + "-ts";
}

private static boolean isTimestampKey(String key) {
    return key.endsWith("-ts");
}

private static String extractShopper(String key) {

```

```

    }

    private static String asCartKey(String shopper) {
        return shopper + "-cart";
    }

    private void resetShopper(String shopper) {
        store.delete(asTimestampKey(shopper));
        store.delete(asCartKey(shopper));
    }
}

```

- **1 For an add-to-cart event, fetch the current cart for this shopper from the key-value store, add this new item to the cart, and then write the updated cart back to the key-value store. Update the shopper's last-active timestamp in the key-value store as well.**
- **2 Whenever a shopper places an order, clear any current tracking of their cart.**
- **3 Every 30 seconds, iterate through the entire contents of our key-value store, looking for abandoned carts.**
- **4 If a shopper's last active key is more than 30 minutes ago, we have an abandoned shopping cart.**
- **5 Sends a "Shopper abandons cart" event to the derived-events-ch05 stream. The event was generated using shopper and cart information held in the key-value store.**
- **6 Helper to reverse out a shopper's ID from a key in the key-value store**

There's a lot to take in with this Samza job: let's briefly review how our new job's `process()` and `window()` functions work. To start with `process()`:

- We are interested in *only Shopper places order* and *Shopper adds item to cart* events.
- When a *Shopper adds item to cart*, we update a copy of their cart stored in our key-value store and update our shopper's last active timestamp.
- When a *Shopper places order*, we delete all state about our shopper from the key-value store.

Our `process()` function is responsible for keeping a copy of each shopper's cart up-to-date based on their add-to-cart events; it is also responsible for understanding how recently the user added something to their cart.

Now let's briefly recap the `window()` function:

- Every 30 seconds, we scan the whole key-value store, looking for shoppers who were *last* active more than 30 minutes ago.
- We generate a *Shopper abandons cart* event for each shopper we find, detailing the contents of their shopping cart as recorded in our key-value store.
- We send each *Shopper abandons cart* event to our outbound Kafka stream.
- We delete from the key-value all values for the shoppers who just abandoned their carts.

With the code complete, let's now compile and package our new Samza job. Still from the project root, run the following:

```
$ mvn clean package
...
[INFO] Building tar: .../nile-carts/target/nile-carts-0.14.0-dist.tar.gz
[INFO] -----
--
[INFO] BUILD SUCCESS
...
```

Great—everything compiles, and we have now packaged our first Samza job. We are ready to run our job on a resource management framework called Apache Hadoop YARN. We'll cover this in the next section.

5.5. Running our Samza job

Although Samza now supports being embedded into a regular JVM application (like Kafka Streams), the more common way of running a Samza job is via YARN. Unless you have previously worked with Hadoop, it's unlikely that you will have encountered YARN before—so we will introduce YARN briefly before getting our job running on it.

5.5.1. Introducing YARN

YARN stands for *Yet Another Resource Negotiator*—an uncomplimentary backronym for an important piece of technology. YARN is a software system that evolved out of Hadoop 1. The biggest difference between Hadoop 1 and Hadoop 2 is the separation of the cluster management responsibility into the YARN subproject.

YARN is deployed onto a Hadoop cluster to allocate resources to YARN-aware applications effectively. It has three core components:

- **ResourceManager**— The central “brain” that tracks servers in the Hadoop cluster and jobs running on those servers, and allocates compute resources to these jobs

- **NodeManager**— Runs on every server in the Hadoop cluster, monitoring the jobs and reporting back to the ResourceManager
- **ApplicationMaster**— Runs alongside each application and negotiates the required resources from the ResourceManager, and works with the NodeManager to execute and monitor each task

YARN is somewhat unfashionable these days—the Kubernetes project gets much more attention—but it is tried and tested technology (rather like ZooKeeper is), and it is widely deployed, given the pervasiveness of Hadoop 2 environments. YARN was also designed in a generic-enough way that various stream processing frameworks, completely unrelated to Hadoop, have been able to use it as their job scheduler. Samza falls into this bucket.

That's enough theory; now we need to install YARN. Fortunately for us, the Hello World project for Samza comes with a script called `grid` that helps you to set up YARN, as well as Kafka and ZooKeeper. This script will also check out the correct version of Samza and build it. You worked with Kafka and ZooKeeper in the preceding chapter—but `grid` will set up these too if they are not still running. All the new software will be added into a subdirectory called `deploy` inside the root folder.

From the project root, run the `grid` script like so:

```
$ bin/grid bootstrap
Bootstrapping the system...
EXECUTING: stop kafka
...
kafka has started
```

You can now navigate to the YARN UI in your web browser:

```
http://localhost:8088
```

The list of All Applications will be empty. We will change this in the next section, by submitting our Samza job to YARN.

5.5.2. Submitting our job

Make sure that you are still in the root folder of your project, and then run the following:

```
$ mkdir -p deploy/samza
$ tar -xvf ./target/nile-carts-0.14.0-dist.tar.gz -C deploy/samza
```

We can now submit our Samza job to YARN:

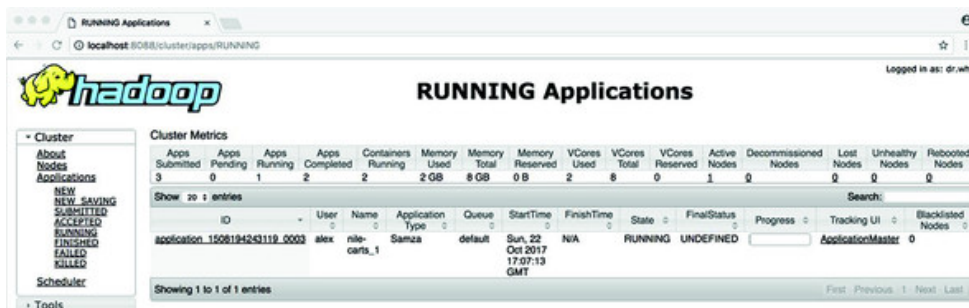
```
$ deploy/samza/bin/run-app.sh \
  --config-factory=org.apache.samza.config.factories.PropertiesConfigFact
  --config-path=file://$PWD/deploy/samza/config/nile-carts.properties
...
2018-10-15 17:25:30.434 [main] JobRunner [INFO] job started successfully
  - Running
2018-10-15 17:25:30.434 [main] JobRunner [INFO] exiting
```

If you return to the YARN UI, you should be able to see your Samza jobs running, as in [figure 5.10](#):

`http://localhost:8088`

Now let's put our new job through its paces.

Figure 5.10. The YARN UI is now showing our Samza job in RUNNING state.



5.5.3. Testing our job

To test our job, we will send e-commerce events into a new Kafka topic, `raw-events-ch05`, and then look for events written by our Samza job into a new stream, this time called `derived-events-ch05`. If our Samza job is working, this output stream will receive fully formed *subject-verb-object* events.

Let's start by *tailing* our output stream. From the root of the project folder, run this command:

```
$ deploy/kafka/bin/kafka-console-consumer.sh \
  --topic derived-events-ch05 --from-beginning \
  --bootstrap-server localhost:9092
```

In a separate terminal, let's start a script that lets us send events into our Kafka topic for incoming events, which we will call `raw-events-ch05`. Samza has automatically created this Kafka topic for us, thanks to its mention in the Samza job configuration. Start up the event producer like so:


```
$ deploy/kafka/bin/kafka-console-producer.sh --topic raw-events-ch05 \
--broker-list localhost:9092
```

Let's start by sending in a couple of add-to-basket events from different shoppers:

```
{ "subject": { "shopper": "123" }, "verb": "add", "indirectObject":
  "cart", "directObject": { "item": { "product": "aabattery", "quantity":
    12 } }, "context": { "timestamp": "2018-10-25T11:56:00" } }

{ "subject": { "shopper": "456" }, "verb": "add", "indirectObject":
  "cart", "directObject": { "item": { "product": "macbook", "quantity":
    1 } }, "context": { "timestamp": "2018-10-25T11:56:12" } }
```

Now here's the thing: by the time you run these commands, these add-to-basket events will be long in the past—far more than 30 minutes ago. If you switch back to your `derived-events-ch05` stream, you should see your first generated events very soon:

```
{"subject":{"shopper":"123"},"verb":"abandon","context":{"timestamp":
  "2018-10-25T11:56:00"},"direct-object":{"cart":{"items":
  [{"product":"aabattery","quantity":12}]}}}}
{"subject":{"shopper":"456"},"verb":"abandon","context":{"timestamp":
  "2018-10-25T11:56:12"},"direct-object":{"cart":{"items":
  [{"product":"macbook","quantity":1}]}}}}
```

So far, so good; next let's try an add-to-basket followed quickly by a check-out event:

```
{ "subject": { "shopper": "789" }, "verb": "add", "indirectObject":
  "cart", "directObject": { "item": { "product": "skyfall", "quantity":
    1 } }, "context": { "timestamp": "timestamp":"2018-10-25T12:00:00" } }

{ "subject": { "shopper": "789" }, "verb": "place", "directObject": {
  "order": { "id": "123", "value": 12.99, "items": [ { "product":
    "skyfall", "quantity": 1 } ] } }, "context": { "timestamp":
    "2018-10-25T12:01:00" } }
```

Assuming you managed to send in both events within the same 30-second window, you should expect no *Shopper abandons cart* event in `derived-events-ch05`. This is because shopper 789 placed their order within the allowed time.

And that's it: our Samza job is monitoring a stream of incoming events to detect abandoned shopping carts. You can try it out with a few more

events of your own creation—just be careful to follow the existing schema.

From the project root, you can now run the following command in order to stop all processes:

```
$ bin/grid stop all
EXECUTING: stop all
EXECUTING: stop kafka
EXECUTING: stop yarn
stopping resourcemanager
stopping nodemanager
EXECUTING: stop zookeeper
```

5.5.4. Improving our job

This Samza job is a good first stab at an abandoned cart detector. If you have time, you could enhance and extend it in a variety of ways:

- De-duplicate items in the cart recorded in Samza’s key-value store. If a shopper adds one copy of the *Skyfall* DVD to their basket twice, this is rationalized to one item with a quantity of 2.
- Use the timestamps of incoming events (from the event context) to determine when the shopper was last active, rather than basing it on the time when `process()` is running, as now.
- Define a new *Shopper removes item from cart* event, and use these events to remove items from the shopper’s cart as stored in Samza’s key-value store.
- Base each shopper’s last-active timestamp on all events for that shopper, not just *Shopper adds item to cart* events.
- Explore more-sophisticated ways of determining whether a cart is abandoned, instead of the strict 30-minute cutoff. You could try varying the cutoff based on the shopper’s previously observed behaviors.

Summary

- Processing multiple events from a stream requires state. This state allows the app to “remember” important attributes about individual events, across many events.
- State for event stream processing can be kept in-memory (transient), stored locally on the processing instance, or written to a remote database. A stream processing framework can help us to manage this process.
- Stream processing frameworks also help us to apply time windows to our streams, have delivery guarantees for our events, distribute our stream processing across multiple servers, and tolerate failures in those individual servers and processes.

- Popular stream-processing frameworks include Apache Storm, Apache Samza, Spark Streaming, Kafka Streams, and Apache Flink.
- Samza is a stateful stream-processing framework with a relatively low-level API, making it the “Hadoop MapReduce of stream processing.”
- Samza’s `process()` function lets us update values in our key-value store every time we get a new event from our incoming event stream in Kafka.
- Samza’s `window()` function lets us regularly review what has changed in our application.
- By clever data modeling in our key-value store (for example, with composite keys), we can detect sophisticated patterns and behaviors, such as shopping-cart abandonment.

[Support](#) [Sign Out](#)